

D1 Control Tower: live code, each followed by its flowchart

The eight exhibits E1 to E8 as plain, step-numbered Python against the current Anthropic SDK with claude-sonnet-5-5 as the main model and claude-haiku-4-5 for small jobs. After each file comes a colour flowchart of the same code, with the points that matter for the CCAR-P exam marked by red dashed callouts. The files are the same as **d1/live/** in the kaveri-dock-lab repository.

How to read the flowcharts

Colour	Means
Yellow box	Code setting things up: constants, schemas, the tool functions. The model never sees these.
Green box	A model call. Each one is paid for. Count the green boxes to count the bill.
Blue box	A code step: building the transcript, running a tool, counting, a plain-Python gate.
Red diamond	A decision on one field, usually stop_reason or a word against a set.
Purple box	A result, or a hand-off to a human.
Red dashed callout with a star	An exam point: the fact, law, or distractor a stem is likely to test.

Before you run anything

```
pip install anthropic
export ANTHROPIC_API_KEY=sk-ant-...    # or copy .env.example to .env in the repo root; never commit it
cd d1/live
python e1_live.py
```

All eight files compile. E1 and E7 were run against the repository's dry-run stand-in and produced the expected traces. None has been run against the live API yet. Compare the stop_reason trace and the control flow; the wording will vary.

Contents

1. E1 The augmented LLM (e1_live.py) code, then flowchart
2. E2 Prompt chaining (e2_live.py) code, then flowchart
3. E3 Routing (e3_live.py) code, then flowchart
4. E4 Parallelization (e4_live.py) code, then flowchart
5. E5 Orchestrator-workers (e5_live.py) code, then flowchart
6. E6 Evaluator-optimizer (e6_live.py) code, then flowchart
7. E7 The agent loop (e7_live.py) code, then flowchart
8. E8 Re-planning (e8_live.py) code, then flowchart

E1 The augmented LLM: the code

One cell. One call, one tool, at most one round-trip.

```
# e1_live.py
# E1: ask the model to sort one customer email into a category.
# The model may ask us to look up the shipment first. If it does, we look it up and tell it the answer.
#
# Before running:  pip install anthropic
#                  put ANTHROPIC_API_KEY in your environment (or in a .env file that is git-ignored)
# Run:             python e1_live.py

import json
import anthropic

# -----
# STEP 0. Things we decide before talking to the model
# -----
MODEL = "claude-sonnet-5-5"      # which model answers
MAX_TOKENS = 1024                # the longest reply we will accept

# The customer's email. This is the only input.
EMAIL = """Subject: KF-2481 - where is my shipment?? Three days late!

Hello Kaveri,

My consignment KF-2481 (Pune to Chennai, 12 cartons of textile samples) was promised on Monday. It is now
Thursday. Your portal says "in transit" and nothing else. My buyer is threatening to cancel the order.

Where is it right now, and what are you doing about it?
- Padma Deshmukh
"""

# The instructions we tape to the model's desk.
SYSTEM = (
    "You triage Kaveri shipment exceptions. "
    "Reply as JSON {category, urgency}. "
    "Categories: delay, damage, customs, refund, address, other."
)

# A description of ONE tool the model is allowed to ask for.
# This is only a description. The model cannot run it. We run it.
TOOLS = [
    {
        "name": "get_shipment_status",
        "description": "Live status for one Kaveri consignment id (e.g. KF-2481).",
        "input_schema": {
            "type": "object",
            "properties": {"consignment_id": {"type": "string"}},
            "required": ["consignment_id"],
        },
    },
]

# -----
# STEP 1. The tool itself. Plain Python. The model never sees this code.
# In real life this would read the depot database.
# -----

def get_shipment_status(consignment_id):
    return {
        "id": consignment_id,
        "status": "CUSTOMS_HOLD",
        "missing": "e-way bill",
        "days_late": 3,
    }

# -----
# STEP 2. Connect to the API. The key is read from ANTHROPIC_API_KEY.
# -----

client = anthropic.Anthropic()

# -----
# STEP 3. Start the conversation with one message: the email.
# -----

conversation = []
```

```

conversation.append({"role": "user", "content": EMAIL})
# -----
# STEP 4. First call to the model.
# -----

reply = client.messages.create(
    model=MODEL,
    max_tokens=MAX_TOKENS,
    system=SYSTEM,
    tools=TOOLS,
    messages=conversation,
)
print("Turn 1 finished because:", reply.stop_reason)

# -----
# STEP 5. Did the model ask for the tool?
# "tool_use" means: "I need a lookup before I can answer."
# -----

if reply.stop_reason == "tool_use":

    # 5a. Find the tool request inside the reply.
    # A reply is a list of blocks. We want the block whose type is "tool_use".
    tool_request = None
    for block in reply.content:
        if block.type == "tool_use":
            tool_request = block
            break
    print("Model asked for tool:", tool_request.name)
    print("With input:", tool_request.input)

    # 5b. Run the tool ourselves, with the input the model filled in.
    consignment_id = tool_request.input["consignment_id"]
    lookup_result = get_shipment_status(consignment_id)
    print("Tool returned:", lookup_result)

    # 5c. Add the model's own request to the conversation.
    # The API remembers nothing between calls, so we must send its own words back.
    conversation.append({"role": "assistant", "content": reply.content})

    # 5d. Add the tool's answer to the conversation, tagged with the request's id,
    # so the model knows which question this answers.
    tool_answer = {
        "type": "tool_result",
        "tool_use_id": tool_request.id,
        "content": json.dumps(lookup_result),      # the API wants text, so turn the dict into a JSON string
    }
    conversation.append({"role": "user", "content": [tool_answer]})
    # 5e. Second call. Same settings. The conversation is now 3 messages long.
    reply = client.messages.create(
        model=MODEL,
        max_tokens=MAX_TOKENS,
        system=SYSTEM,
        tools=TOOLS,
        messages=conversation,
    )

    print("Turn 2 finished because:", reply.stop_reason)

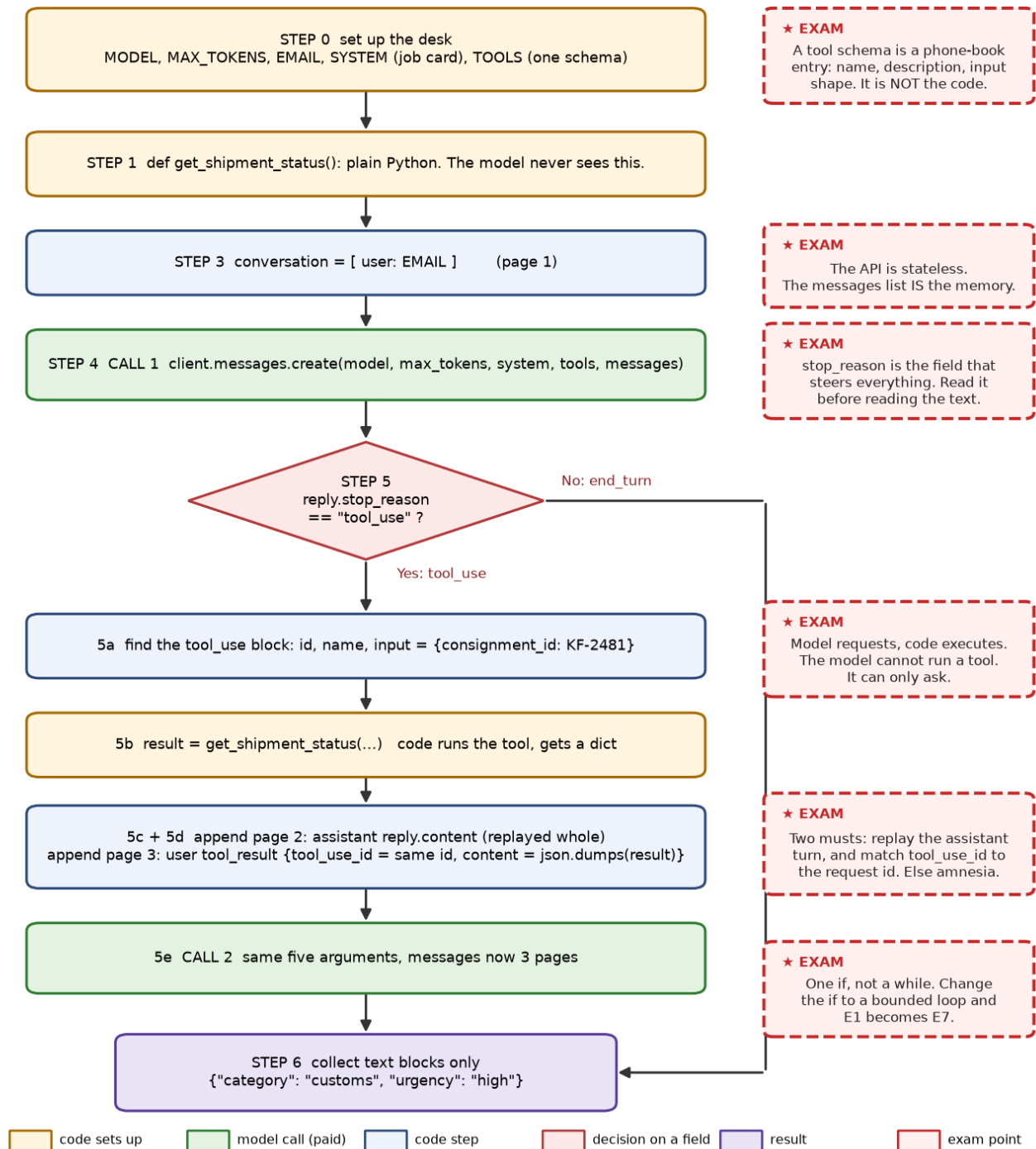
# -----
# STEP 6. Read the final answer.
# -----
if reply.stop_reason == "refusal":
    print("The model declined to answer.")
else:
    # Collect only the text blocks. Skip anything else, such as thinking blocks.
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text

    print("Final answer:", answer)

```

E1 The augmented LLM: the flowchart

E1 The augmented LLM: one cell, at most one round-trip



Exam points on this chart

- ★ A tool schema is a description, not code. The model requests; your code executes.
- ★ The API is stateless: the messages list is the memory. Replay the assistant turn and match tool_use_id.
- ★ stop_reason steers everything. E1 uses one if; E7 turns that if into a bounded loop.

E2 Prompt chaining: the code

Two desks in a fixed order with a plain-Python gate between them.

```
# e2_live.py
# E2: prompt chaining. Two desks in a fixed order, with a checklist between them.
# Desk 1: read the email, say ONE word: what kind of problem is this?
# Gate: plain Python checks that word is on the approved list. If not, stop.
# Desk 2: knowing the category, write a short courteous reply.
# The CODE decides the order. The model never gets to change it.
#
# Before running: pip install anthropic
#                  put ANTHROPIC_API_KEY in your environment (or in a git-ignored .env file)
# Run:             python e2_live.py

import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MODEL = "claude-sonnet-5-5"          # one model for both desks, to keep it simple
MAX_TOKENS = 1024

# The customer's email. Same one as E1.
EMAIL = """Subject: KF-2481 - where is my shipment?? Three days late!

Hello Kaveri,
My consignment KF-2481 (Pune to Chennai, 12 cartons of textile samples) was promised on Monday. It is now
  Thursday. Your portal says "in transit" and nothing else. My buyer is threatening to cancel the order.

Where is it right now, and what are you doing about it?

- Padma Deshmukh
"""

# The laminated checklist on the wall between the two desks.
# Only these six words are allowed to pass from desk 1 to desk 2.
VALID_CATEGORIES = {"delay", "damage", "customs", "refund", "address", "other"}
# -----
# STEP 1. Connect to the API. The key is read from ANTHROPIC_API_KEY.
# -----

client = anthropic.Anthropic()

# -----
# STEP 2. A small helper: send one question, get the text back.
#         No tools here. E2 is text in, text out, twice.
# -----

def ask(question):
    reply = client.messages.create(
        model=MODEL,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    # If the model declined, do not pretend the empty text is an answer.
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")

    # Collect only the text blocks. Skip thinking blocks or anything else.
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text

    return answer.strip()
# -----
# STEP 3. DESK 1: classify. One word only.
# -----

def classify(email_text):
    question = (
        "Read this customer email and reply with ONE word only, the category of the problem. "
        "Choose from: delay, damage, customs, refund, address, other. "
        "No explanation, no punctuation, just the word.\n\n"
        + email_text
    )
```

```

    word = ask(question)
    return word.lower()          # make "Customs" and "customs" the same thing
# -----
# STEP 4. THE GATE: plain Python, no model.
#     If desk 1 mumbled something that is not on the checklist, stop the line here.
# -----

def gate(category):
    if category not in VALID_CATEGORIES:
        raise ValueError("Bad category from desk 1: " + repr(category))
    return category

# -----
# STEP 5. DESK 2: draft the reply. It is TOLD the category; it does not re-decide it.
# -----
def draft_reply(email_text, category):
    question = (
        "This is a " + category + " exception at Kaveri Freight. "
        "Draft a courteous reply to the customer, under 120 words. "
        "Do not promise any refund or compensation.\n\n"
        + email_text
    )
    return ask(question)

# -----
# STEP 6. Run the line, in the order the code fixes: classify -> gate -> draft.
# -----
print("=== Desk 1: classify ===")
category = classify(EMAIL)
print("Desk 1 said:", repr(category))

print()
print("=== Gate: check the word against the list ===")
category = gate(category)
print("Gate passed:", category, "is on the approved list. Line continues.")

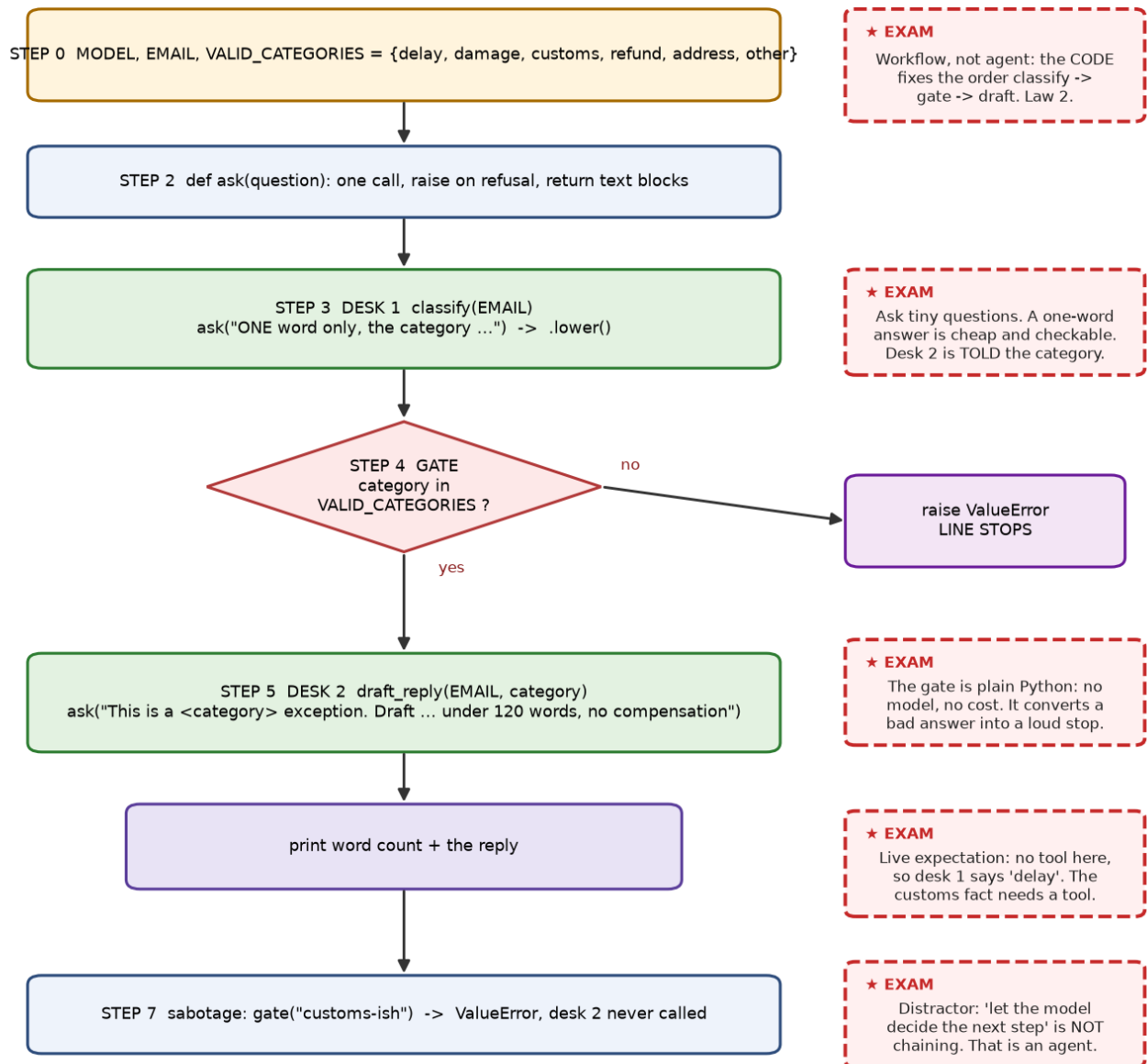
print()
print("=== Desk 2: draft ===")
reply = draft_reply(EMAIL, category)
word_count = len(reply.split())
print("Desk 2 wrote", word_count, "words:")
print()
print(reply)
# -----
# STEP 7. Sabotage test. What happens if desk 1 had mumbled?
#     No model call here. We hand the gate a bad word on purpose.
# -----

print()
print("=== Sabotage: feed the gate a word that is not on the list ===")
try:
    gate("customs-ish")
except ValueError as error:
    print("Gate stopped the line:", error)
    print("Desk 2 was never called. No reply was written for a nonsense category.")

```

E2 Prompt chaining: the flowchart

E2 Prompt chaining: fixed order, a code gate between the desks



code sets up model call (paid) code step decision on a field result exam point

Exam points on this chart

- ★ Workflow, not agent: the code fixes the order. Law 2.
- ★ Ask tiny questions; a one-word answer is cheap and checkable. Desk 2 is told the category.
- ★ The gate costs nothing and converts a bad answer into a loud stop.

E3 Routing: the code

A cheap nurse call picks the ward; the ward picks the model.

```
# e3_live.py
# E3: routing. A triage nurse at the door, specialist wards behind it.
# Nurse: one cheap call says ONE word: which kind of problem is this?
# Gate: plain Python checks the word is on the list.
# Ward: the word picks WHICH specialist (and which model) handles the email. One call.
# The model that answers the customer never saw a menu. The code chose the ward.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e3_live.py

import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MAX_TOKENS = 1024

# Three model sizes. Routing is where "which model" becomes a decision PER category.
MODEL_BIG = "claude-sonnet-5-5" # for money decisions (refunds). In production: the Opus tier.
MODEL_MID = "claude-sonnet-5-5" # for explanations (customs)
MODEL_SMALL = "claude-haiku-4-5" # for short templated notes (delay apologies) and the nurse

EMAIL = """Subject: KF-2481 - where is my shipment?? Three days late!
Hello Kaveri,

My consignment KF-2481 (Pune to Chennai, 12 cartons of textile samples) was promised on Monday. It is now
Thursday. Your portal says "in transit" and nothing else. My buyer is threatening to cancel the order.

Where is it right now, and what are you doing about it?

- Padma Deshmukh
"""

VALID_CATEGORIES = {"delay", "damage", "customs", "refund", "address", "other"}

# The wards. Each category maps to (which model, what instructions).
# This table IS the router. Add a ward by adding a row. No model is involved in the choice.
SPECIALISTS = {
    "refund": (MODEL_BIG, "You resolve refund disputes at Kaveri Freight. Acknowledge, say what you will
        verify, never promise money."),
    "customs": (MODEL_MID, "You explain customs holds and next steps for Indian import consignments at Kaveri
        Freight."),
    "delay": (MODEL_SMALL, "You write short delay apologies for Kaveri Freight with a new ETA. Under 80
        words."),
}
# If the nurse says a word that has no ward (for example "address"), send it here instead of crashing.
DEFAULT_WARD = "delay"

# -----
# STEP 1. Connect, and one helper: ask a model a question, get text back.
# This time the helper takes the MODEL as an argument, because each ward uses a different one.
# -----

client = anthropic.Anthropic()

def ask(model, question):
    reply = client.messages.create(
        model=model,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer.strip()
# -----
# STEP 2. The nurse (same as E2's desk 1) and the gate (same as E2's gate).
# -----

def classify(email_text):
```

```

question = (
    "Read this customer email and reply with ONE word only, the category of the problem. "
    "Choose from: delay, damage, customs, refund, address, other. "
    "No explanation, no punctuation, just the word.\n\n"
    + email_text
)
return ask(MODEL_SMALL, question).lower()
def gate(category):
    if category not in VALID_CATEGORIES:
        raise ValueError("Bad category from the nurse: " + repr(category))
    return category

# -----
# STEP 3. The router: word in, specialist reply out.
# -----

def route(email_text):
    # 3a. The nurse looks once and says one word.
    category = gate(classify(email_text))
    # 3b. Plain Python looks the word up in the table. .get() with a default means
    #      an unmapped word goes to a safe ward instead of raising KeyError at 2 a.m.
    if category in SPECIALISTS:
        model, instructions = SPECIALISTS[category]
    else:
        model, instructions = SPECIALISTS[DEFAULT_WARD]

    print("Nurse said:", repr(category))
    print("Routed to ward:", category if category in SPECIALISTS else DEFAULT_WARD, "using model:", model)

    # 3c. ONE call to the chosen specialist. It gets its own instructions plus the email.
    return ask(model, instructions + "\n\n" + email_text)

# -----
# STEP 4. Run it.
# -----

print("=== Routing Padma's email ===")
specialist_reply = route(EMAIL)
print()
print("Specialist wrote:")
print(specialist_reply)

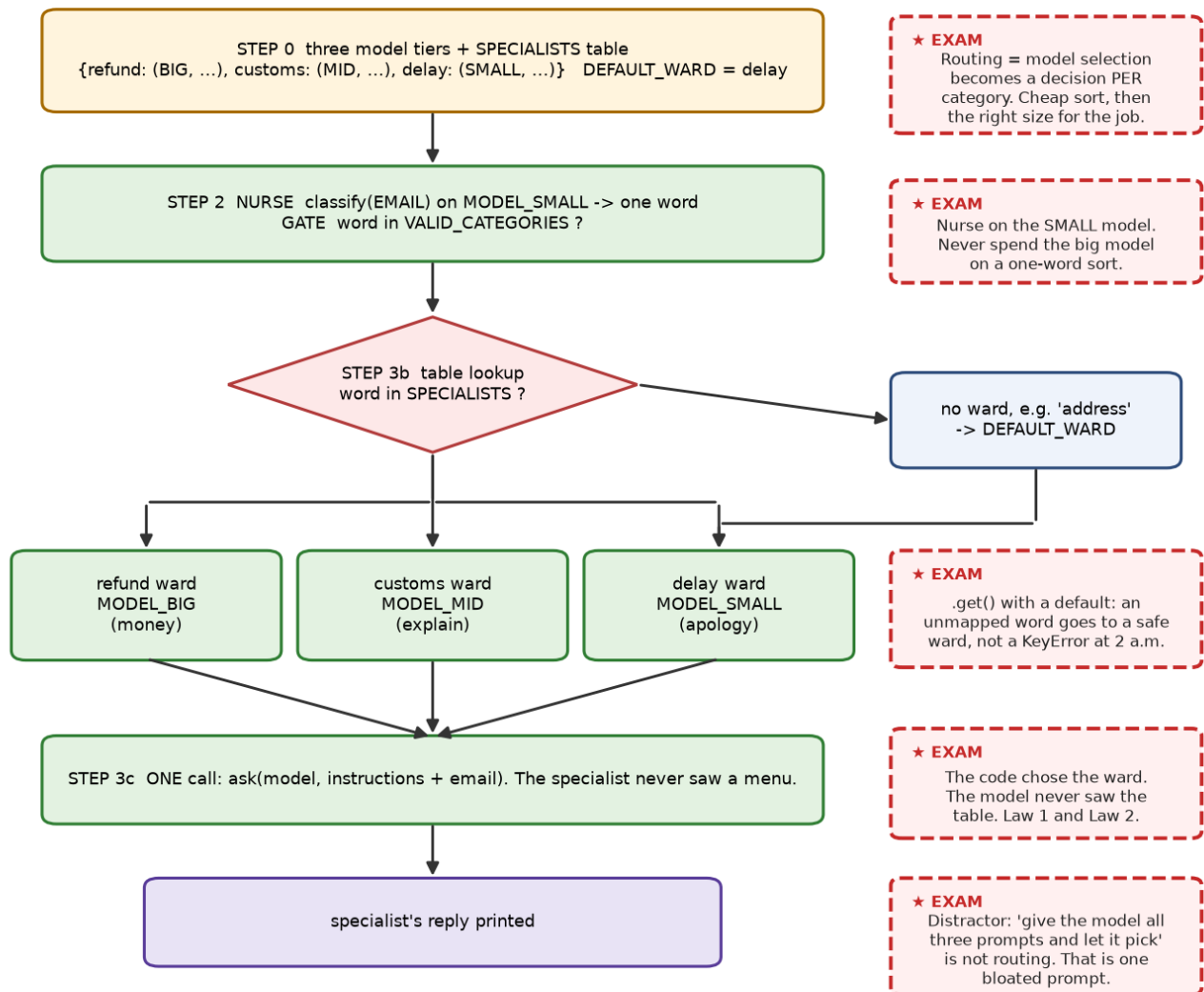
# -----
# STEP 5. Show the default ward. No model call: just the table lookup.
# -----

print()
print("=== What happens to a word with no ward ===")
word = "address"
if word in SPECIALISTS:
    chosen = word
else:
    chosen = DEFAULT_WARD
print(repr(word), "has no ward, so it goes to", repr(chosen), "using model", SPECIALISTS[chosen][0])

```

E3 Routing: the flowchart

E3 Routing: a cheap nurse picks the ward, the ward picks the model



code sets up model call (paid) code step decision on a field result exam point

Exam points on this chart

- ★ Routing is where model selection becomes a decision per category.
- ★ Nurse on the small model; never spend the big model on a one-word sort.
- ★ A default ward via .get(): an unmapped word is a safe fallback, not a KeyError.

E4 Parallelization: the code

Sectioning for coverage, voting for confidence. Same trick, opposite purposes.

```
# e4_live.py
# E4: parallelization. Same trick (several calls at the same time), two opposite purposes.
# Sectioning: three DIFFERENT questions at once (three scouts on three roads). Purpose: coverage, speed.
# Voting: the SAME question three times (three judges). Purpose: confidence. Costs 3x.
# The code drew the fork. No model decided to parallelize.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e4_live.py

import asyncio
import time
import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MODEL = "claude-sonnet-5-5"
MAX_TOKENS = 256 # short answers are all we need here

CONSIGNMENT = "KF-2481"
CARRIERS = ["BlueDart", "Delhivery", "IndiaPost"]

# The refund claim the three judges will vote on.
CLAIM = """Refund claim: consignment KF-1177, declared value Rs 18,500.
Two cartons water-damaged on arrival at the Bengaluru hub. Customer requests a full refund.
Photos attached show a tear in the outer packaging; inner goods wet. Carrier: Delhivery.
Insurance: declared-value cover active. Delivery was signed for without a damage note.
"""
# -----
# STEP 1. Connect with the ASYNC client. "async" lets several calls be in flight at once.
# -----

client = anthropic.AsyncAnthropic()

async def ask(question):
    reply = await client.messages.create(
        model=MODEL,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer.strip()
# -----
# STEP 2. SECTIONING: three different questions, all at once.
# -----

async def check_one_carrier(carrier):
    question = (
        "In one line, give a plausible ETA for consignment " + CONSIGNMENT
        + " on the Pune-Chennai lane if re-routed via " + carrier + ". "
        "Assume it is currently held at customs. Start your line with the carrier name."
    )
    return await ask(question)

async def sectioning():
    # asyncio.gather starts all three and waits for the slowest. Wall clock = slowest scout.
    return await asyncio.gather(
        check_one_carrier(CARRIERS[0]),
        check_one_carrier(CARRIERS[1]),
        check_one_carrier(CARRIERS[2]),
    )

async def sequential():
    # The same three questions one after another, for comparison. Wall clock = the sum.
    results = []
    for carrier in CARRIERS:
        results.append(await check_one_carrier(carrier))
    return results
# -----
# STEP 3. VOTING: the same question three times, majority wins.
```

```

# -----

async def one_vote():
    question = (
        "You are a claims reviewer at Kaveri Freight. Read this refund claim and reply with "
        "exactly one word, APPROVE or REJECT. No explanation.\n\n" + CLAIM
    )
    return await ask(question)

async def voting():
    votes = await asyncio.gather(one_vote(), one_vote(), one_vote())
    # Count the votes in plain Python.
    approve_count = 0
    reject_count = 0
    for vote in votes:
        if "APPROVE" in vote.upper():
            approve_count = approve_count + 1
        else:
            reject_count = reject_count + 1

    if approve_count > reject_count:
        majority = "APPROVE"
    else:
        majority = "REJECT"
    return votes, majority

# -----
# STEP 4. Run both, and time the sectioning against the sequential version.
# -----

async def main():
    print("=== Sectioning: three scouts at once ===")
    start = time.perf_counter()
    scouts = await sectioning()
    parallel_seconds = time.perf_counter() - start
    for line in scouts:
        print("  scout ->", line)
    print()
    print("=== The same three scouts, one after another ===")
    start = time.perf_counter()
    await sequential()
    sequential_seconds = time.perf_counter() - start

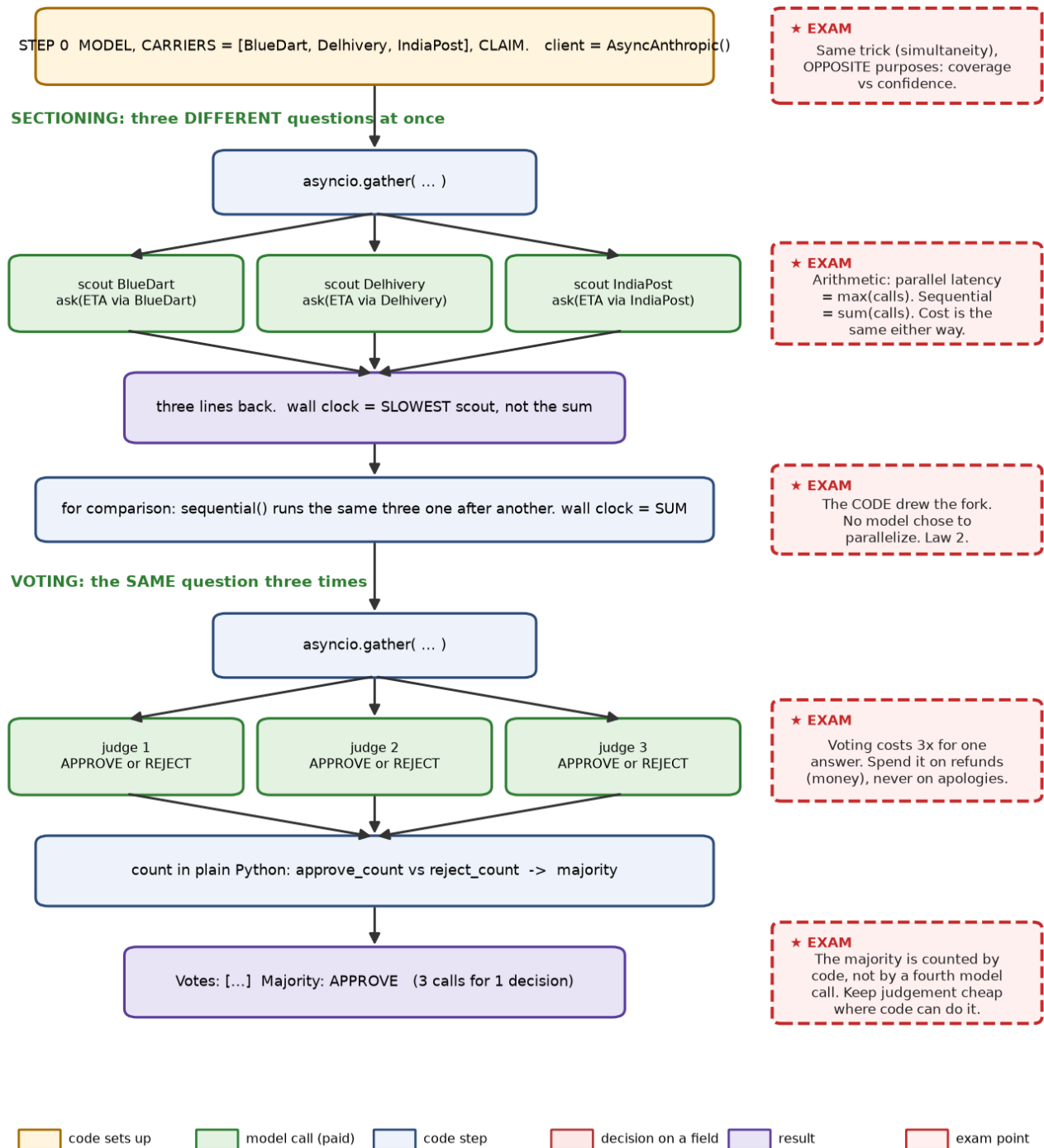
    print("Parallel took", round(parallel_seconds, 2), "s (about the slowest single call).")
    print("Sequential took", round(sequential_seconds, 2), "s (about the sum of all three).")

    print()
    print("=== Voting: three judges on one refund claim ===")
    votes, majority = await voting()
    print("Votes:", votes)
    print("Majority:", majority)
    print("Note: this cost three calls for one decision. Spend that on refunds, not on apologies.")
    asyncio.run(main())

```

E4 Parallelization: the flowchart

E4 Parallelization: sectioning for coverage, voting for confidence



Exam points on this chart

- ★ Parallel latency = max(calls); sequential = sum(calls). Cost is the same either way.
- ★ Voting costs 3x for one decision: spend it on money, not on apologies.
- ★ The code drew the fork and the code counts the votes.

E5 Orchestrator-workers: the code

The foreman model writes the plan; code steers the workers; synthesis writes one recommendation.

```
# e5_live.py
# E5: orchestrator-workers. The site foreman writes the plan himself.
# Foreman (model): reads the brief, writes a list of subtasks as JSON.
# Workers (model): one call per subtask. Each sees only the findings it declared it needs.
# Synthesis (model): sees all findings, writes ONE recommendation.
# The SHAPE (plan -> work -> synthesise) is fixed code. The CONTENT of the plan is the model's.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e5_live.py

import json
import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MODEL_FOREMAN = "claude-sonnet-5-5" # planning and synthesis need judgement
MODEL_WORKER = "claude-haiku-4-5" # each subtask is small and focused
MAX_TOKENS = 1024

BRIEF = """KF-2481's customer (Padma Deshmukh) is threatening to leave. The consignment is three days late on
the Pune-Chennai lane. Find out why it is stuck and recommend what Meera's desk should do today."""
# -----
# STEP 1. Connect, and two helpers: ask for text, and ask for JSON.
# -----

client = anthropic.Anthropic()

def ask(model, question):
    reply = client.messages.create(
        model=model,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer.strip()

def ask_json(model, question):
    raw = ask(model, question)

    # Models sometimes wrap JSON in a ```json ... ``` fence. Strip it if present.
    if raw.startswith("```"):
        lines = raw.split("\n")
        lines = lines[1:] # drop the opening ```json line
        if lines and lines[-1].strip() == "```":
            lines = lines[:-1] # drop the closing ``` line
        raw = "\n".join(lines)

    return json.loads(raw)
# -----
# STEP 2. The foreman writes the plan.
# -----

def make_plan(brief):
    question = (
        "Brief:\n" + brief + "\n\n"
        "Break this into 2 to 5 subtasks. Reply with JSON only, a list like:\n"
        '["id": 1, "task": "status check", "needs": []], {"id": 2, "task": "carrier options", "needs": [1]}\n'
        '"needs" lists the ids of earlier subtasks whose findings this one depends on.'
    )
    return ask_json(MODEL_FOREMAN, question)
# -----
# STEP 3. One worker does one subtask. It is shown ONLY the findings it declared it needs.
# -----

def run_worker(subtask, findings_so_far):
    # Collect just the findings this subtask asked for.
    known = {}
    for needed_id in subtask["needs"]:
        if needed_id in findings_so_far:
```

```

        known[needed_id] = findings_so_far[needed_id]

    question = (
        "You are a worker at Kaveri Freight. Do this one subtask and reply in one or two lines.\n"
        "Subtask: " + subtask["task"] + "\n"
        "Known facts from earlier subtasks: " + json.dumps(known) + "\n"
        "Context: consignment KF-2481, Pune-Chennai, three days late. "
        "If you have no data, state the most likely finding for a late Indian domestic consignment."
    )
    return ask(MODEL_WORKER, question)
# -----
# STEP 4. Synthesis sees everything and writes one recommendation.
# -----

def synthesise(brief, findings):
    question = (
        "Brief:\n" + brief + "\n\n"
        "Findings from the workers:\n" + json.dumps(findings, indent=2) + "\n\n"
        "Write ONE recommendation for Meera's desk, three sentences at most, starting with RECOMMENDATION:"
    )
    return ask(MODEL_FOREMAN, question)
# -----
# STEP 5. Run the three stages in the fixed order.
# -----

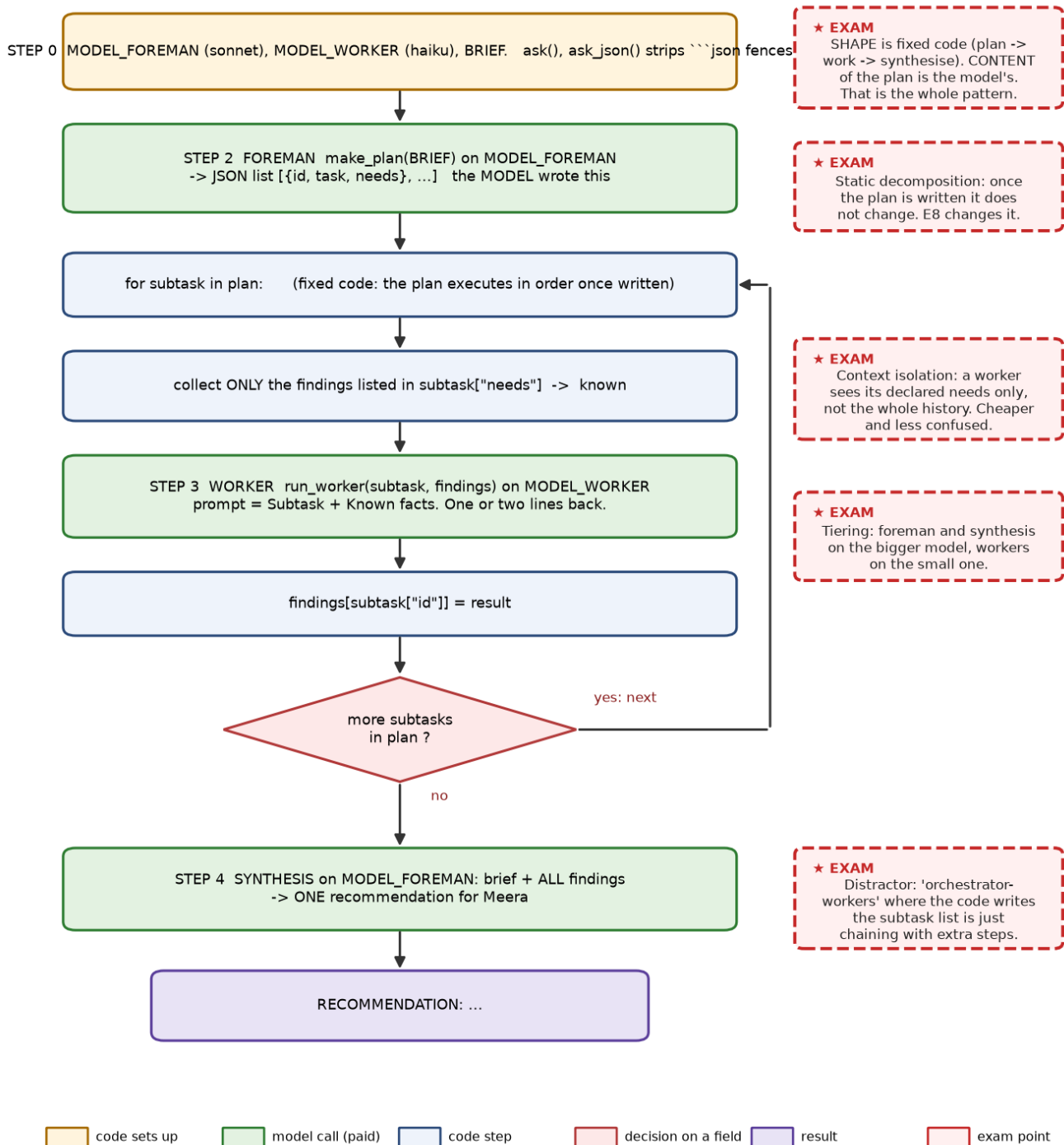
print("=== Foreman writes the plan ===")
plan = make_plan(BRIEF)
print(json.dumps(plan, indent=2))

print()
print("=== Workers run, one per subtask, in plan order ===")
findings = {}
for subtask in plan:
    result = run_worker(subtask, findings)
    findings[subtask["id"]] = result
    print("Worker", subtask["id"], "(" + subtask["task"] + ") needs", subtask["needs"], "->", result)
print()
print("=== Synthesis ===")
print(synthesise(BRIEF, findings))

```

E5 Orchestrator-workers: the flowchart

E5 Orchestrator-workers: the foreman writes the plan, code steers the work



Exam points on this chart

- ★ Shape is fixed code; content of the plan is the model's. Static decomposition.
- ★ Context isolation: a worker sees only its declared needs.
- ★ Tiering: foreman and synthesis on the bigger model, workers on the small one.

E6 Evaluator-optimizer: the code

Writer and editor, with a page limit.

```
# e6_live.py
# E6: evaluator-optimizer. A writer and an editor, with a page limit.
# Writer (model): drafts a reply.
# Editor (model): checks it against a rubric. Says PASS, or one line of criticism.
# Writer again: revises using the criticism. Repeat.
# Two safety organs: max_rounds (the budget) and PASS (the exit). E7's agent loop has exactly these two, grown
  larger.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e6_live.py

import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MODEL = "claude-sonnet-5-5"
MAX_TOKENS = 1024
MAX_ROUNDS = 2 # the budget: the editor gets at most two looks

EMAIL = """Subject: KF-2481 - where is my shipment?? Three days late!

Hello Kaveri,
My consignment KF-2481 (Pune to Chennai, 12 cartons of textile samples) was promised on Monday. It is now
  Thursday. Your portal says "in transit" and nothing else. My buyer is threatening to cancel the order.

Where is it right now, and what are you doing about it?

- Padma Deshmukh
"""

RUBRIC = (
    "You are an editor at Kaveri Freight. Check the draft below against this rubric:\n"
    " 1. Tone courteous?\n"
    " 2. Facts only, no promises of refunds or compensation?\n"
    " 3. Under 120 words?\n"
    "If all three pass, reply with exactly the single word PASS. "
    "Otherwise reply with ONE line of specific criticism and nothing else.\n\n"
    "Draft:\n"
)
# -----
# STEP 1. Connect, and the helper.
# -----

client = anthropic.Anthropic()

def ask(question):
    reply = client.messages.create(
        model=MODEL,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer.strip()
# -----
# STEP 2. The loop: draft, then (evaluate, maybe revise) up to MAX_ROUNDS times.
# -----

def draft_and_polish(email_text):
    # 2a. First draft. Deliberately loose instructions so the editor has something to catch.
    draft = ask("Reply to this customer exception email on behalf of Kaveri Freight:\n\n" + email_text)
    print("Draft 0:", len(draft.split()), "words")

    # 2b. The editing rounds. range(MAX_ROUNDS) is the budget.
    for round_number in range(1, MAX_ROUNDS + 1):
        verdict = ask(RUBRIC + draft)
        print("Round", round_number, "editor said:", verdict)
        # The exit. PASS means stop early, do not spend the remaining budget.
        if verdict == "PASS":
```

```

        break

    # Not PASS: revise using the criticism.
    draft = ask(
        "Revise the draft below. Apply this criticism exactly and change nothing else:\n"
        + verdict + "\n\nDraft:\n" + draft
    )
    print("Round", round_number, "revised:", len(draft.split()), "words")

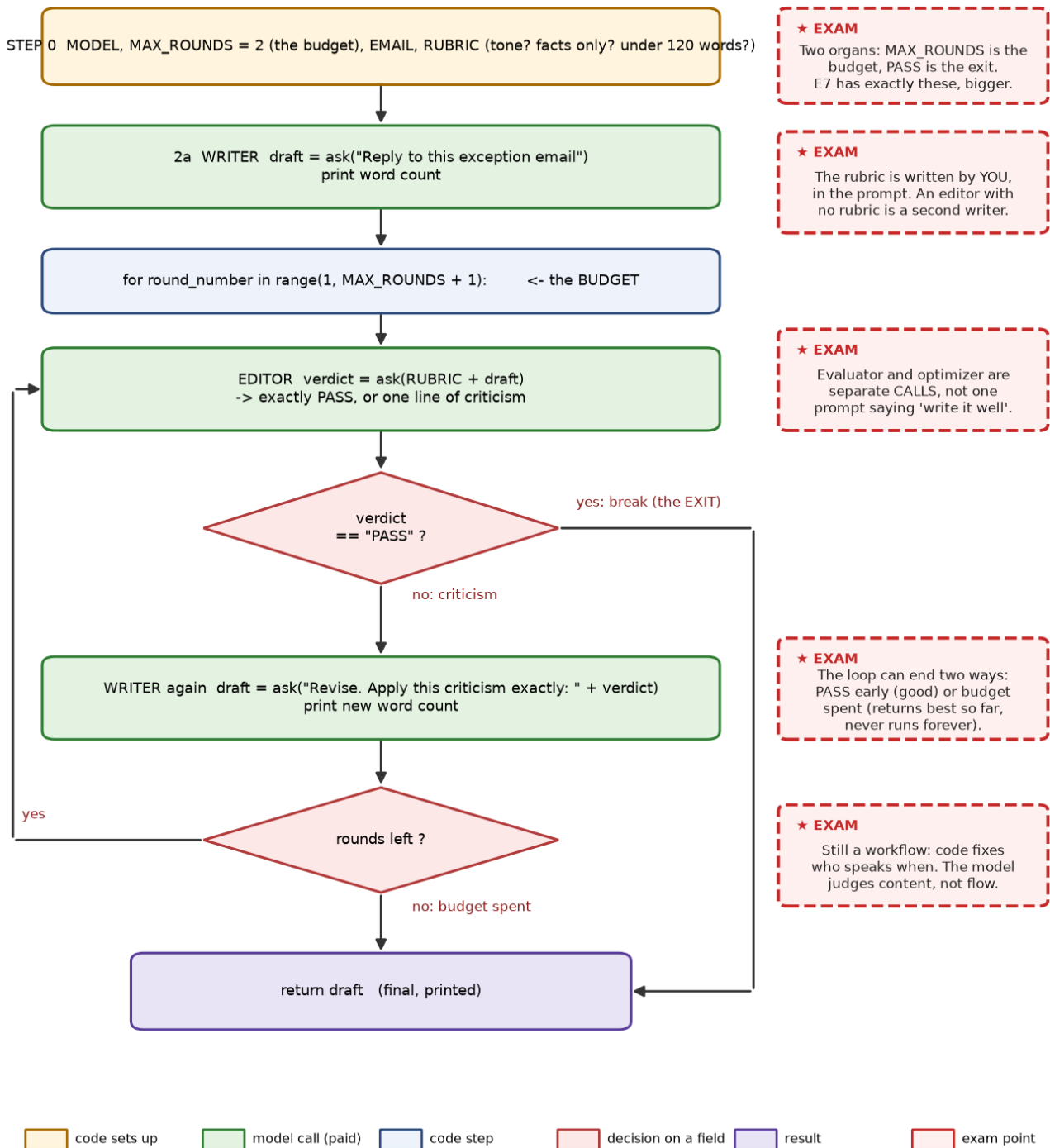
    return draft
# -----
# STEP 3. Run it.
# -----

final = draft_and_polish(EMAIL)
print()
print("=== Final draft ===")
print(final)

```

E6 Evaluator-optimizer: the flowchart

E6 Evaluator-optimizer: writer, editor, a budget and an exit



Exam points on this chart

- ★ Two organs: MAX_ROUNDS is the budget, PASS is the exit. E7 has exactly these, grown larger.
- ★ The rubric is yours and lives in the prompt. Evaluator and optimizer are separate calls.
- ★ The loop ends two ways: PASS early, or budget spent with the best draft so far.

E7 The agent loop: the code

A bounded loop steered by stop_reason, every value on its own branch.

```
# e7_live.py
# E7: the agent loop. A loop around messages.create that keeps going while the model asks for tools
# and stops when it does not. The field that steers it is stop_reason. EVERY value lands somewhere explicit.
# BUDGET is written before the loop exists. Escalation to a human is the floor.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e7_live.py

import json
import anthropic

# -----
# STEP 0. Things we decide before the loop exists
# -----
MODEL = "claude-sonnet-5-5"
MAX_TOKENS = 1024
BUDGET = 6 # the most turns we will ever pay for. Never "while True".

BRIEF = "Why is consignment KF-2481 stuck, and what are our options? Use the tools, then give a short recommendation."

# Two tools the detective may ask for. Descriptions only; we run them.
TOOLS = [
    {
        "name": "get_shipment_status",
        "description": "Live status for one Kaveri consignment id (e.g. KF-2481).",
        "input_schema": {
            "type": "object",
            "properties": {"consignment_id": {"type": "string"}},
            "required": ["consignment_id"],
        },
    },
    {
        "name": "find_alternate_carriers",
        "description": "Carriers that can take a lane (e.g. Pune-Chennai) within 48 hours.",
        "input_schema": {
            "type": "object",
            "properties": {"lane": {"type": "string"}},
            "required": ["lane"],
        },
    },
]

# -----
# STEP 1. The tools themselves. Plain Python. Swap for real lookups in production.
# -----

def get_shipment_status(consignment_id):
    return {"id": consignment_id, "status": "CUSTOMS_HOLD", "missing": "e-way bill", "days_late": 3}

def find_alternate_carriers(lane):
    return ["BlueDart", "Delhivery"]

def run_tool(tool_request):
    # Return a structured error for an unknown tool, not an exception. The model can read it and adapt.
    if tool_request.name == "get_shipment_status":
        return get_shipment_status(tool_request.input["consignment_id"])
    if tool_request.name == "find_alternate_carriers":
        return find_alternate_carriers(tool_request.input["lane"])
    return {"error": "unknown tool " + tool_request.name}

# -----
# STEP 2. The floor. When the loop cannot finish, a human gets the case WITH a reason.
# -----

def escalate(reason):
    return "ESCALATED to Meera's desk (" + reason + "): " + BRIEF

# -----
# STEP 3. Connect.
# -----
client = anthropic.Anthropic()
# -----
# STEP 4. The loop.
```

```

# -----

def investigate():
    conversation = [{"role": "user", "content": BRIEF}]      # the case file. The API is stateless, so this IS
    the memory.
    text_so_far = ""                                       # text that arrived before a max_tokens cut

    for turn in range(1, BUDGET + 1):                      # bounded, always
        reply = client.messages.create(
            model=MODEL,
            max_tokens=MAX_TOKENS,
            tools=TOOLS,
            messages=conversation,
        )

        # One line per turn so you can watch the loop breathe.
        tool_names = []
        for block in reply.content:
            if block.type == "tool_use":
                tool_names.append(block.name)
        print("Turn", turn, "stop_reason =", reply.stop_reason, tool_names if tool_names else "")
        # Collect any text in this reply.
        text_this_turn = ""
        for block in reply.content:
            if block.type == "text":
                text_this_turn = text_this_turn + block.text

        # ---- Every stop_reason value gets its own branch. No "else" that swallows. ----

        if reply.stop_reason == "tool_use":
            # The loop's fuel. Run every tool the model asked for, send all results back.
            conversation.append({"role": "assistant", "content": reply.content})
            results = []
            for block in reply.content:
                if block.type == "tool_use":
                    results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": json.dumps(run_tool(block)),
                    })
            conversation.append({"role": "user", "content": results})
            continue

        if reply.stop_reason == "end_turn":
            # The natural exit. Return everything, including any text saved from a max_tokens cut.
            return text_so_far + text_this_turn

        if reply.stop_reason == "max_tokens":
            # Truncated is not done. Keep the partial text, ask the model to continue.
            text_so_far = text_so_far + text_this_turn
            conversation.append({"role": "assistant", "content": reply.content})
            conversation.append({"role": "user", "content": "Continue."})
            continue

        if reply.stop_reason == "pause_turn":
            # A server-side tool paused the turn. Send the reply back as-is to resume.
            conversation.append({"role": "assistant", "content": reply.content})
            continue

        if reply.stop_reason == "refusal":
            # The model declined. A different path, never a blind retry.
            return escalate("model refused")

        # Anything else: stop_sequence, context window exceeded, a value that does not exist yet.
        return escalate("unhandled stop_reason " + str(reply.stop_reason))

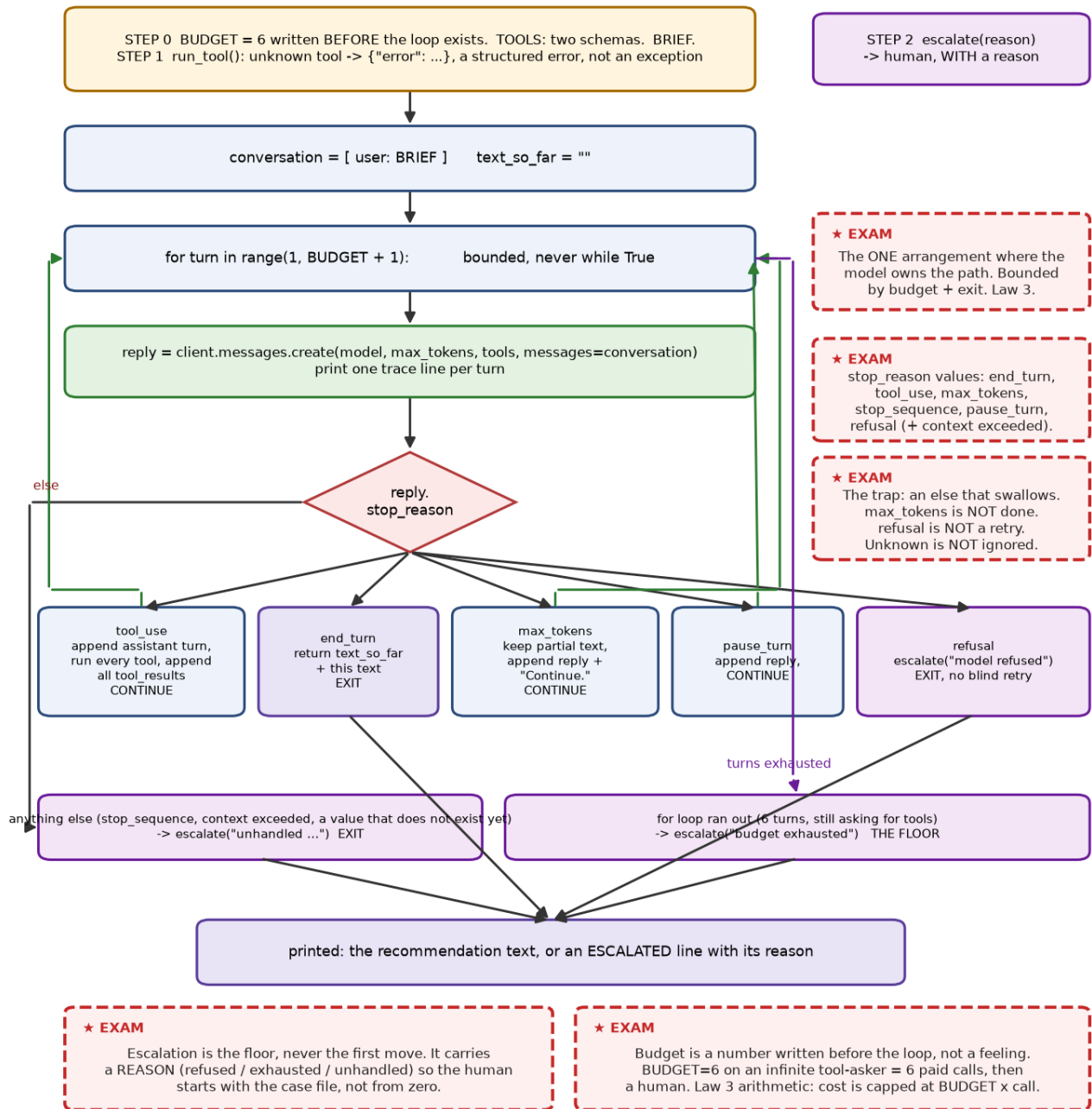
    # The for loop ran out. The case is still open. A human takes it, with the reason.
    return escalate("budget of " + str(BUDGET) + " turns exhausted")

# -----
# STEP 5. Run it.
# -----
print("=== Agent loop, BUDGET =", BUDGET, "===")
print()
print(investigate())

```

E7 The agent loop: the flowchart

E7 The agent loop: every stop_reason lands somewhere explicit



code sets up model call (paid) code step decision on a field result exam point

Exam points on this chart

- ★ The one arrangement where the model owns the path, bounded by budget and exit. Law 3.
- ★ Values: end_turn, tool_use, max_tokens, stop_sequence, pause_turn, refusal, and context exceeded.
- ★ The trap is an else that swallows. max_tokens is not done, refusal is not a retry, escalation carries a reason.

E8 Re-planning: the code

E5 plus a check after every finding: does the remaining plan still hold?

```
# e8_live.py
# E8: re-planning. The bridge is out: a finding can change the plan.
# Same three stages as E5 (plan -> work -> synthesise), plus after EVERY finding the foreman is asked:
# "does the remaining plan still hold?" If not, drop some tasks and add others.
# A second budget, MAX_NEW, caps how many tasks the plan may grow by.
#
# Before running: pip install anthropic and ANTHROPIC_API_KEY in the environment
# Run: python e8_live.py

import json
import anthropic
# -----
# STEP 0. Things we decide before talking to the model
# -----

MODEL_FOREMAN = "claude-sonnet-5-5"
MODEL_WORKER = "claude-haiku-4-5"
MAX_TOKENS = 1024
MAX_NEW = 3 # the growth budget: at most 3 tasks may be added, ever

BRIEF = """KF-2481's customer (Padma Deshmukh) is threatening to leave. The consignment is three days late on
the Pune-Chennai lane. Find out why it is stuck and recommend what Meera's desk should do today."""
# -----
# STEP 1. Connect, and the two helpers (same as E5).
# -----

client = anthropic.Anthropic()

def ask(model, question):
    reply = client.messages.create(
        model=model,
        max_tokens=MAX_TOKENS,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer.strip()

def ask_json(model, question):
    raw = ask(model, question)
    if raw.startswith("```"):
        lines = raw.split("\n")[1:]
        if lines and lines[-1].strip() == "```":
            lines = lines[:-1]
        raw = "\n".join(lines)
    return json.loads(raw)

# -----
# STEP 2. The three E5 moves, unchanged.
# -----

def make_plan(brief):
    question = (
        "Brief:\n" + brief + "\n\n"
        "Break this into 2 to 5 subtasks. Reply with JSON only, a list like:\n"
        '["id": 1, "task": "status check", "needs": []], {"id": 2, "task": "carrier options", "needs": [1]}'
    )
    return ask_json(MODEL_FOREMAN, question)

def run_worker(subtask, findings_so_far):
    known = {}
    for needed_id in subtask["needs"]:
        if needed_id in findings_so_far:
            known[needed_id] = findings_so_far[needed_id]
    question = (
        "You are a worker at Kaveri Freight. Do this one subtask and reply in one or two lines.\n"
        "Subtask: " + subtask["task"] + "\n"
        "Known facts from earlier subtasks: " + json.dumps(known) + "\n"
        "Context: consignment KF-2481, Pune-Chennai, three days late. "
        "If asked for status, report: CUSTOMS_HOLD at Chennai depot, missing e-way bill. "
        "Otherwise state the most likely finding."
    )
```

```

    )
    return ask(MODEL_WORKER, question)
def synthesise(brief, findings):
    question = (
        "Brief:\n" + brief + "\n\nFindings:\n" + json.dumps(findings, indent=2)
        + "\n\nWrite ONE recommendation for Meera's desk, three sentences at most, starting with
          RECOMMENDATION:"
    )
    return ask(MODEL_FOREMAN, question)

# -----
# STEP 3. The new move: after a finding, ask whether the remaining plan still holds.
#         The foreman sees the new finding and the REMAINING tasks, not the whole history.
# -----
def check_plan(finding, remaining_tasks):
    question = (
        "A worker just reported this finding:\n" + finding + "\n\n"
        "These subtasks are still to be done:\n" + json.dumps(remaining_tasks) + "\n\n"
        "Does the remaining plan still make sense given the finding? Reply with JSON only:\n"
        ' {"ok": true}\n'
        'or\n'
        ' {"ok": false, "drop": [ids of tasks that no longer make sense], "add": ["short name of a new task",
          ...]}'
    )
    return ask_json(MODEL_FOREMAN, question)
# -----
# STEP 4. The loop with re-planning.
# -----
def replan(brief):
    tasks = make_plan(brief)
    print("Initial plan:", json.dumps(tasks))
    print()

    findings = {}
    tasks_added = 0
    next_id = max(task["id"] for task in tasks) + 1
    while tasks:
        current = tasks.pop(0)
        result = run_worker(current, findings)
        findings[current["id"]] = result
        print("Run", current["id"], "(" + current["task"] + ") ->", result)

        verdict = check_plan(result, tasks)

        if verdict.get("ok", True):
            continue
        # plan still holds, move on

        if tasks_added >= MAX_NEW:
            print(" Replan requested, but the growth budget is spent. Keeping the plan as is.")
            continue
        # Drop the tasks the foreman says no longer make sense.
        drop_ids = set(verdict.get("drop", []))
        tasks = [task for task in tasks if task["id"] not in drop_ids]

        # Add the new ones, but never past the cap.
        room_left = MAX_NEW - tasks_added
        new_names = verdict.get("add", [])[:room_left]
        for name in new_names:
            tasks.append({"id": next_id, "task": name, "needs": []})
            next_id = next_id + 1
        tasks_added = tasks_added + len(new_names)

    print(" REPLAN: dropped", sorted(drop_ids), "added", new_names)
    print()
    print("Tasks added in total:", tasks_added, "of a budget of", MAX_NEW)
    return synthesise(brief, findings)

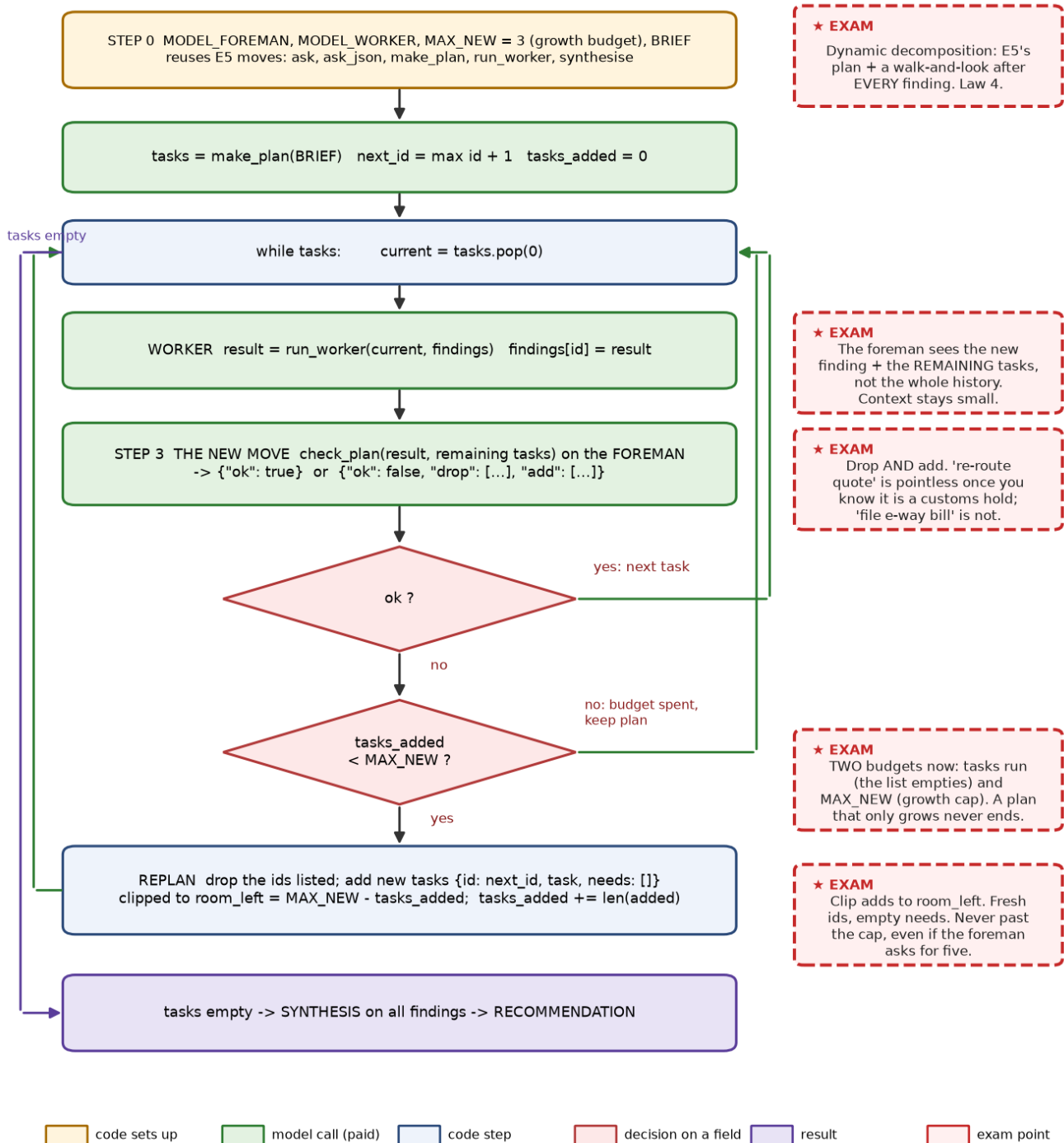
# -----
# STEP 5. Run it.
# -----

print("=== Re-planning run ===")
print()
print(replan(BRIEF))

```

E8 Re-planning: the flowchart

E8 Re-planning: a finding can change the plan, inside a growth budget



Exam points on this chart

- ★ Dynamic decomposition. Law 4. The foreman sees the new finding and the remaining tasks only.
- ★ Drop and add, with fresh ids and empty needs.
- ★ Two budgets: the task list empties, and MAX_NEW caps growth. A plan that only grows never ends.